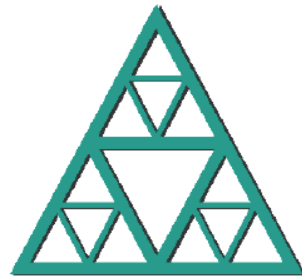


ECOLE NATIONALE DES PONTS ET CHAUSSEES



PROJET DE PREMIÈRE ANNÉE 2025

RAPPORT FINAL

TRUCAGE PHOTOGRAPHIQUE PAR INPAINTING

Paul Andreis et Antoine Salomon

sous la direction de
Pascal Monasse
Laboratoire d'Informatique Gaspard Monge

Key words

Inpainting, Image Completion, Patch-Based Algorithm, Criminisi Algorithm, Image Restoration

1 Abstract

In the field of photo editing, it is common to want to remove an unwanted element from an image. To preserve the visual coherence of the whole, it is then necessary to replace this area with a plausible background. This crucial step is called inpainting.

In this article, we study one of the many inpainting methods by implementing the algorithm proposed by A. Criminisi, P. Perez, and K. Toyama. We begin with an overview of the different inpainting methods, then we examine the functioning of the algorithm by Criminisi et al.[1], its computer implementation, its efficiency, and possible improvements for our implementation.

Table des matières

1	Abstract	1
2	Méthodes utilisées en inpainting	2
2.1	Inpainting utilisant les équations de diffusion	2
2.2	Méthodes basées sur l'apprentissage automatique (deep learning)	3
2.3	Méthodes basées sur des Patches	3
3	Présentation de l'algorithme	3
3.1	Notations	3
3.2	Pseudo-code de l'algorithme	3
3.3	Sélection du prochain patch à remplir (Ψ_c)	4
3.3.1	Terme de confiance $C(p)$	4
3.3.2	Terme de données $D(p)$	4
3.4	Sélection du patch Ψ_p minimisant la distance à $\Psi_c \cap \Phi$	5
3.5	Copie de Ψ_p sur Ψ_c	5
3.6	Calcul du gradient et de la normale à la frontière	6
3.6.1	Calcul du gradient	6
3.6.2	Calcul de la normale à la frontière	6
4	Implémentation	7
4.1	Architecture du code	7
4.2	Algorithme de Criminisi	7
4.3	Analyse temporelle de la fonction <code>criminisi()</code>	7
4.4	Détails d'implémentation	8
4.5	Sélection de l'espace Φ	9
5	Etude des résultats	10
5.1	Performance de l'algorithme	10
5.2	Impact de la taille du patch	12
5.2.1	Impact sur la complexité temporelle	12
5.2.2	Qualité du résultat	13

6 Améliorations de l'algorithme	14
6.1 Espace colorimétrique CIE LAB plutôt que RGB	14
6.2 Approche par optimisation globale	14

2 Méthodes utilisées en inpainting

Tous les algorithmes d'inpainting partent de l'image intacte et tentent de remplir la partie manquante à partir de cette donnée. Pour ce faire différentes options sont possibles :

2.1 Inpainting utilisant les équations de diffusion

Il s'agit de la première méthode d'inpainting numérique, introduite en 2000 par Marcelo Bertalmío, Guillermo Sapiro, Vincent Caselles et Coloma Ballester dans l'article fondateur "*Navier-Stokes, fluid dynamics, and image and video inpainting*" [2]. Cette approche repose sur les équations physiques de diffusion, telles que celles de la chaleur ou des fluides, pour propager l'information de l'image intacte vers les zones manquantes.

Les algorithmes cherchent alors à résoudre, de manière approchée, des équations aux dérivées partielles complexes telles que celle de Navier-Stokes. Cela permet de prolonger la structure de l'image : les lignes d'intensité constante (appelées *isophotes*) sont conservées, ce qui rend le remplissage localement cohérent avec le reste de l'image.

Cependant, cette méthode présente des limites lorsqu'il s'agit de remplir de grandes zones manquantes : l'inpainting par diffusion tend à produire des résultats flous, comme illustre la figure 1.



(a) Image initiale



(b) Résultat de l'inpainting par diffusion

FIGURE 1 – Résultat d'un trucage par les équations de diffusion (comparaison tirée de [1])

Ainsi, cette méthode n'est efficace que pour la restauration d'images dégradées ou la correction de motifs simples (formes géométriques ou petits défauts). En revanche, pour supprimer des éléments de grande taille, d'autres approches sont généralement préférées.

2.2 Méthodes basées sur l'apprentissage automatique (deep learning)

Ces méthodes sont apparues dans les années 2010. Elles reposent sur des réseaux de neurones profonds (*deep learning*) et constituent aujourd'hui les approches les plus prometteuses dans le domaine. Leur principe repose sur un apprentissage supervisé à partir d'un jeu de données d'images partiellement masquées. Les réseaux utilisés sont généralement des réseaux de neurones convolutifs (appelés *CNN*). L'une des premières applications notables de cette approche a été proposée par Pathak *et al.* [3] en 2016. Le grand avantage de cette méthode est sa capacité à en extraire une représentation sémantique. Ces réseaux sont ainsi capables de combler de larges zones tout en conservant la cohérence structurelle de l'image.

Plus récemment, des modèles comme celui de Chen *et al.* [4] exploitent une architecture antagoniste combinant un générateur — chargé de compléter l'image — et un discriminateur — chargé d'évaluer le réalisme du résultat. Cette opposition entre les deux réseaux permet d'obtenir des remplissages visuellement très convaincants.

2.3 Méthodes basées sur des Patches

C'est cette méthode qui sera utilisée dans notre étude. Popularisée par Criminisi *et al.* [1] en 2004, elle consiste à copier dans la zone manquante des morceaux entiers (*patches*) extraits de la partie intacte de l'image. Contrairement aux méthodes par diffusion, elle ne produit pas d'effet de flou, et se montre particulièrement efficace pour prolonger des motifs réguliers ou des textures répétitives.

3 Présentation de l'algorithme

Pour réaliser notre algorithme d'inpainting par copie de patches nous nous sommes appuyés sur la méthode décrite par Criminisi et al.[1]. Bien que cet algorithme ait été développé par A. Criminisi, P. Pérez et K. Toyama, il est généralement désigné dans la littérature sur l'inpainting comme «l'algorithme de Criminisi». Nous l'appellerons donc ainsi dans la suite.

3.1 Notations

Nous adopterons les notations usuelles de la littérature sur l'inpainting. La région déjà remplie de l'image sera notée Φ , tandis que la région à remplir sera notée Ω . Ces deux régions évoluant au cours de l'exécution de l'algorithme, nous noterons respectivement Φ_0 et Ω_0 leur état initial. Le contour de cette dernière, c'est-à-dire la frontière entre Φ et Ω , sera noté $\delta\Omega$. Un *patch* désigne une petite portion carrée de l'image, de quelques pixels de côté, et sera noté Ψ . On notera abusivement $\Psi_p \in A$ si le centre p du patch Ψ_p appartient à l'ensemble A .

3.2 Pseudo-code de l'algorithme

Voici le pseudo code de l'algorithme :

La sélection de la zone Ω_0 peut se faire ici manuellement par l'utilisateur. L'efficacité de l'algorithme dépend grandement de la méthode utilisée pour les étapes (1) et (2) que nous décrirons dans les sections suivantes.

Entrée : Une image séparée en deux régions (Φ_0 et Ω_0)
 Sortie : Image remplie en une seule région : Φ
 Pseudo-code :
 Tant que ($\Omega \neq \emptyset$) :
 Sélectionner $\Psi_c \in \delta\Omega$ (1)
 Sélectionner $\Psi_p \in \Phi$ qui minimise la *distance* à $\Psi_c \cap \Phi$ (2)
 Copier Ψ_p sur Ψ_c (3)

3.3 Sélection du prochain patch à remplir (Ψ_c)

L'ordre de remplissage de la zone manquante est particulièrement crucial. En effet, l'algorithme de Criminisi adopte une approche gloutonne : une fois un patch placé — même s'il est peu cohérent avec le contexte — il n'est jamais modifié par la suite. Ce fonctionnement peut conduire à une propagation d'artefacts sur de grandes zones de l'image. Il est donc nécessaire de commencer par remplir les zones les plus simples (structurées ou bien définies), avant de traiter les régions plus complexes.

Pour cela, Criminisi *et al.*[1] introduisent un mécanisme de file de priorité qui permet d'ordonner les patches à traiter. Soit un point $p \in \delta\Omega$, et Ψ_p le patch de centre p , la priorité de ce patch est notée $P(p)$ et se compose sous la forme du produit :

$$P(p) = C(p) \cdot D(p)$$

où $C(p)$ est le *terme de confiance* et $D(p)$ le *terme de données*, qui seront détaillés ci-dessous.

Le calcul de ces deux termes permet alors d'instaurer un ordre total de priorité sur l'ensemble des points $p \in \delta\Omega$. A chaque itération on sélectionne alors $\Psi_c \in \delta\Omega$ où

$$c = \arg \max_{p \in \delta\Omega} P(p)$$

c'est-à-dire le patch de centre c ayant la plus haute priorité.

3.3.1 Terme de confiance $C(p)$

Le terme de confiance reflète la quantité d'information fiable à l'intérieur de patch Ψ_p , un pixel provenant de l'image intacte initiale (ie Φ_0) à un terme de confiance valant 1 tandis qu'un point provenant de Ω_0 peut être considéré comme ayant un terme de confiance nul. Le terme de confiance d'un patch entier peut alors être considéré comme la moyenne des termes de confiance des points le composant :

$$C(p) = \frac{1}{|\Psi_p|} \sum_{q \in \Psi_p \cap \Phi} C(q)$$

Nous décrirons dans la section 3.5 comment sont actualisé les termes de confiance dans les patches nouvellement créés

3.3.2 Terme de données $D(p)$

Pour maintenir le réalisme d'une image, il est essentiel de préserver les contours et les structures géométriques présentes dans l'image. Pour ce faire, le terme de données favorise les patches situés

dans des zones où l'information structurelle est la plus forte. Ce terme est donc plus élevé au niveau des isophotes (c'est-à-dire les lignes de même intensité).

Il est défini comme suit :

$$D(p) = \frac{|\nabla I_p^\perp \cdot n_p|}{\alpha}$$

où :

- $\nabla I_p^\perp$ est le vecteur orthogonal au gradient de l'image à l'intérieur de Φ , approximant la direction de l'isophote arrivant en p ,
- n_p est le vecteur normal à la frontière $\delta\Omega$ au point p , pointant vers l'intérieur de la région à remplir,
- $\cdot$ désigne le produit scalaire,
- α est un facteur de normalisation (par exemple $\alpha = 255$ pour des images sur 8 bits).

3.4 Selection du patch Ψ_p minimisant la distance à $\Psi_c \cap \Phi$

Une fois le patch de plus haute priorité trouvé, il est nécessaire de déterminer le patch de l'image intacte qui lui est le plus similaire (noté Ψ_p).

Pour cela, on introduit la distance suivante entre deux patches :

$$d(\Psi_a, \Psi_b) = \sqrt{\sum_{q \in \Psi_a \cap \Phi} \sum_{c \in \{R, G, B\}} (I_c(q) - I_c(q + (b - a)))^2}$$

Ce qui correspond à la *Sum of Squared Difference (SSD)* entre les pixels déjà remplis des deux patches. Ainsi, on peut déterminer parmi l'ensemble des pixels de l'image celui qui minimise cette distance ; on choisit alors :

$$\Psi_p = \arg \min_{\Psi \in \Phi} d(\Psi_c, \Psi)$$

Le patch Ψ_p est alors le patch visuellement le plus proche de Ψ_c .

3.5 Copie de Ψ_p sur Ψ_c

Une fois ces deux patches sélectionnés par les étapes précédentes, il suffit de recopier les pixels manquants du patch Ψ_c , c'est-à-dire les pixels appartenant à l'intersection $\Psi_c \cap \Omega$, en copiant les valeurs correspondantes depuis le patch Ψ_p .

Il est aussi nécessaire d'actualiser le terme de confiance de chaque pixel du patch Ψ_c , afin de refléter la fiabilité des nouveaux pixels ajoutés. Pour cela, tous les pixels nouvellement remplis (c'est-à-dire $q \in \Psi_c \cap \Omega$) se voient attribuer la même valeur de confiance que le centre du patch traité, p :

$$\forall q \in \Psi_c \cap \Omega, \quad C(q) \leftarrow C(p)$$

Ainsi, à chaque itération, l'ensemble des pixels remplis se voit attribuer une valeur de confiance, laquelle sera réutilisée lors des itérations suivantes.

3.6 Calcul du gradient et de la normale à la frontière

Un détail clé de notre algorithme, utilisé lors du calcul du terme de données, est la méthode de calcul du gradient et de la normale à la frontière que nous allons détailler ci-dessous.

3.6.1 Calcul du gradient

Dans notre implémentation, le calcul du gradient est effectué exclusivement sur les pixels connus de l'image. Nous utilisons une approche par calcul de dérivées centrées qui permet d'évaluer la variation de l'intensité lumineuse dans les directions x et y .

Formellement, soit $I(x, y)$ l'intensité de l'image en un pixel (x, y) , et $\mathcal{M}(x, y)$ la fonction de masque où $\mathcal{M}(x, y) = 0$ indique un pixel connu et $\mathcal{M}(x, y) = 1$ un pixel masqué.

Les directions de voisinage considérées sont définies par l'ensemble de vecteurs :

$$\mathcal{D} = \{(1, 0), (1, 1), (1, -1), (-1, 0), (-1, 1), (-1, -1), (0, 1), (0, -1)\}$$

Le gradient local $\nabla I(x, y)$ est défini par :

$$\nabla I(x, y) = \begin{cases} \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) & \text{si } \mathcal{M}(x, y) = 0 \\ (NaN, NaN) & \text{si } \mathcal{M}(x, y) = 1 \end{cases}$$

Où l'on estime les dérivées partielles $\frac{\partial I}{\partial x}$ et $\frac{\partial I}{\partial y}$ par une approche pondérée :

$$\begin{aligned} \frac{\partial I}{\partial x} &= \frac{\sum_{(dx, dy) \in \mathcal{D}} \frac{[I(x+dx, y+dy) - I(x, y)] \cdot dx}{\sqrt{dx^2 + dy^2}}}{\sum_{(dx, dy) \in \mathcal{D}} \sqrt{dx^2 + dy^2}} \\ \frac{\partial I}{\partial y} &= \frac{\sum_{(dx, dy) \in \mathcal{D}} \frac{[I(x+dx, y+dy) - I(x, y)] \cdot dy}{\sqrt{dx^2 + dy^2}}}{\sum_{(dx, dy) \in \mathcal{D}} \sqrt{dx^2 + dy^2}} \end{aligned}$$

Les pixels masqués se voient attribuer un gradient de valeur NaN pour qu'ils soient exclus des calculs suivants.

3.6.2 Calcul de la normale à la frontière

Afin de déterminer la direction du vecteur normal en un point p de la frontière $\delta\Omega$, on commence par estimer la tangente en ce point. Pour cela, on sélectionne les points de la frontière situés dans une fenêtre locale (ici un carré de 6 pixels de côté) centrée sur p . En soustrayant les coordonnées de ces points $\{p_i\}$ à celles de p , on obtient des vecteurs $\vec{t}_i = p_i - p$ qui approchent la tangente locale. On moyenne ces vecteurs, afin d'obtenir une estimation plus robuste et lissée de la tangente en p :

$$\vec{t} = \frac{1}{N} \sum_{i=1}^N \vec{t}_i.$$

Il suffit ensuite d'appliquer une rotation de 90 degrés à cette tangente pour obtenir la direction du vecteur normal :

$$\vec{n} = (-t_y, t_x).$$

4 Implémentation

L'ensemble du code a été développé en C++, en s'appuyant sur la bibliothèque graphique `Imagine++` pour la gestion de l'affichage et du chargement d'images.

4.1 Architecture du code

Le programme est structuré autour de trois fichiers principaux :

- `main.cpp` : contient l'interface utilisateur minimale (sélection de la zone à effacer) et le déclenchement de l'algorithme.
- `image.cpp` : regroupe les traitements sur l'image via la classe `CriminisiImage`, qui encapsule les données (image couleur, masque, confiance, gradients, etc.) ainsi que les fonctions associées (calcul de gradients, détection de contour, priorité, etc.).
- `patch.cpp` : définit la classe `Patch`, qui représente un patch centré sur un point donné, et contient les fonctions essentielles à l'algorithme comme le calcul de la SSD (Sum of Squared Differences), la mise à jour, la confiance, et la priorité.

4.2 Algorithme de Criminisi

L'algorithme procède itérativement à remplir une région masquée de l'image comme décrit dans l'algorithme en pseudo code suivant :

```
Entrée : Une image avec région masquée  $\Omega$ , contour  $\delta\Omega$ , taille de
patch  $P$ 
Sortie : Image complétée (inpainting de  $\Omega$ )
Algorithme de Criminisi :
Tant que  $\Omega \neq \emptyset$  et itération < maxIter :
    Calculer les priorités pour tous les points de  $\delta\Omega$  (confiance  $\times$ 
données)
    Sélectionner le point  $p$  de priorité maximale sur le contour
    Construire le patch  $\Psi_p$  centré en  $p$ 
    Chercher le patch source  $\Psi_q$  complètement connu qui minimise la
    SSD avec  $\Psi_p$ 
    Copier les pixels valides de  $\Psi_q$  dans  $\Psi_p$ 
    Mettre à jour les valeurs de confiance dans  $\Psi_p$ 
    Mettre à jour le masque  $\Omega$ , le contour  $\delta\Omega$ , les gradients
    Afficher l'image mise à jour
```

FIGURE 2 – Pseudocode de l'algorithme de Criminisi pour l'inpainting

4.3 Analyse temporelle de la fonction `criminisi()`

La fonction `criminisi()` est le cœur de l'algorithme, où se déroule le remplissage progressif de la région masquée. Elle fonctionne de manière itérative, avec les opérations suivantes à chaque itération :

- `computePriorityList()` : calcule la priorité de chaque point du contour.

Chaque point nécessite la création d'un patch ($\mathcal{O}(P)$), le calcul de la confiance ($\mathcal{O}(P)$), la normale ($\mathcal{O}(1)$) et le produit scalaire avec l'isophote ($\mathcal{O}(1)$). On a finalement un coût total de $\mathcal{O}(C \cdot P)$, où C est le nombre de pixels de contour, et P la taille d'un patch.

- `findPatchList()` : génère tous les patches valides.

On teste chaque pixel (x, y) de l'image comme centre potentiel ($\mathcal{O}(W \cdot H)$), puis on vérifie si tous les pixels dans la fenêtre sont valides ($\mathcal{O}(P)$). Le coût total est donc de $\mathcal{O}(W \cdot H \cdot P)$.

- `ssd()` : compare le patch cible avec tous les patches valides.

Comparaison avec N patches valides, chaque distance coûtant $\mathcal{O}(P)$, ce qui donne un coût temporel total de $\mathcal{O}(N \cdot P)$.

- `updatePatch()` : copie les valeurs du patch source vers la région à remplir.

La mise à jour coûte $\mathcal{O}(P)$.

- `computeMask()` et `findContours()` : mise à jour du masque et du contour.

Le parcours de l'image s'effectue en $\mathcal{O}(W \cdot H)$.

- `show()` : réaffiche l'image.

La conversion de l'image en trois canaux séparés pour l'affichage a un coût de $\mathcal{O}(W \cdot H)$.

Ainsi, pour chaque itération, la complexité est dominée par :

$$\mathcal{O}(W \cdot H \cdot P + N \cdot P + C \cdot P)$$

avec $C \ll W \cdot H$ et $N \approx W \cdot H$ en pratique. La complexité par itération devient donc :

$$\mathcal{O}(W \cdot H \cdot P)$$

Et pour T itérations (généralement proportionnelles au nombre de pixels masqués), la complexité globale est :

$$\mathcal{O}(T \cdot W \cdot H \cdot P)$$

En conclusion, la complexité totale est dominée par la recherche du meilleur patch, soit :

$$\mathcal{O}(T \cdot W \cdot H \cdot P)$$

où T est le nombre d'itérations, généralement borné par le nombre de pixels masqués.

Cette analyse permet de justifier les choix d'optimisation possibles (limitation du nombre d'itérations, réduction de la taille des patches, ou utilisation de stratégies heuristiques pour restreindre l'espace de recherche).

4.4 Détails d'implémentation

Lors de l'implémentation de l'algorithme, nous avons été amenés à faire certains choix plus spécifiques que nous allons détailler dans les paragraphes qui suivent.

Tout d’abord, nous avons choisi de représenter le masque comme une image binaire, où les pixels noirs sont considérés comme masqués. Lorsqu’on lance l’algorithme notamment, la région Ω_0 est représentée en noir.

Pour ce qui concerne les gradients, ils ne sont calculés que sur les pixels connus via des dérivées centrées. Les pixels masqués se voient attribuer un gradient NaN.

Un patch source est valide si et seulement si tous ses pixels sont connus, i.e. que tous ses pixels sont de confiance 1.

Nous avons choisi d’imposer que la taille des patches soit impaire (typiquement 9×9), afin de s’assurer que le patch soit centré en un pixel donné. Cela garantit la symétrie autour du point cible et simplifie les calculs de voisinage.

Enfin, pour ce qui est de l’affichage du résultat, nous avons choisi d’afficher l’image à chaque itération afin de rendre compte du déroulement de l’algorithme. Chaque affichage est suivi d’une très courte pause de sorte que l’image reste visible pendant un certain temps. Cela rallonge le temps d’exécution, mais permet de donner une bonne idée de ce qui se passe lors du déroulement de l’algorithme.

Ces choix, bien qu’arbitraires dans certains cas, ont été guidés par une volonté de clarté et de simplicité dans l’implémentation du projet.

4.5 Sélection de l’espace Φ

En plus de choisir la zone Ω_0 de l’image à remplir, il est possible de modifier l’algorithme afin de restreindre la zone dans laquelle sont recherchés les patches source (Φ_0). Cela présente deux avantages :

Accélération de l’algorithme : la recherche du patch source (étape (2)) est l’opération la plus coûteuse, s’effectuant en $O(W \times H)$, où W et H sont respectivement la largeur et la hauteur de l’image. Réduire la taille de la fenêtre de recherche permet donc un gain de temps significatif.

Amélioration possible de la qualité du résultat : en limitant la zone de recherche, on oriente l’algorithme vers les motifs pertinents à prolonger. Cela permet d’éviter que des motifs provenant d’autres éléments indésirables de l’image ne soient copiés dans la zone à remplir.

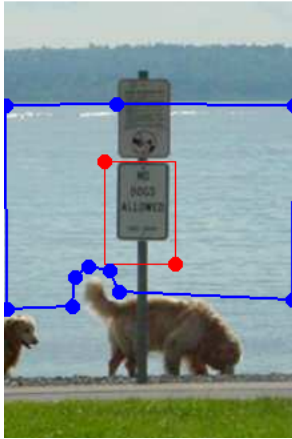
Dans notre cas, nous avons implémenté cette fonctionnalité de la manière suivante : après avoir sélectionné la zone à effacer, l’utilisateur peut cliquer sur différents points de l’image afin de définir un polygone (délimité en bleu, comme illustré ci-dessous). L’intérieur de ce polygone correspond à la zone de recherche Φ_0 .



(a) Image initiale sans sélection



(b) Résultat sans sélection en 43 secondes



(c) Image initiale avec sélection



(d) Résultat avec sélection en 26 secondes

FIGURE 3 – Comparaison des résultats avec et sans sélection (image initiale tirée de [1])

Sur la figure (b), le poteau présente une tache marron, due à une texture provenant de la queue du chien. En sélectionnant manuellement l'eau comme zone de recherche Φ_0 , ce phénomène est évité. Par ailleurs, on remarque que le temps de calcul est presque divisé par deux grâce à cette sélection (43 secondes contre 26). Ainsi, la délimitation manuelle de Φ_0 permet non seulement d'accélérer considérablement l'algorithme, mais aussi d'éviter certaines erreurs de remplissage.

5 Etude des résultats

Toutes les images présentées ont été générées à l'aide de notre implémentation en C++, compilée avec gcc 8.3 en mode release, sur un processeur Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz.

5.1 Performance de l'algorithme

Afin d'évaluer l'efficacité de l'algorithme, nous allons comparer les résultats de celui-ci avec celui réalisé par Criminisi et al.[1].

Apparaissent dans la figure 4, de gauche à droite, les images originales, les résultats obtenus par l'algorithme de Criminisi et al.[1] et les résultats que nous avons obtenus avec notre algorithme.

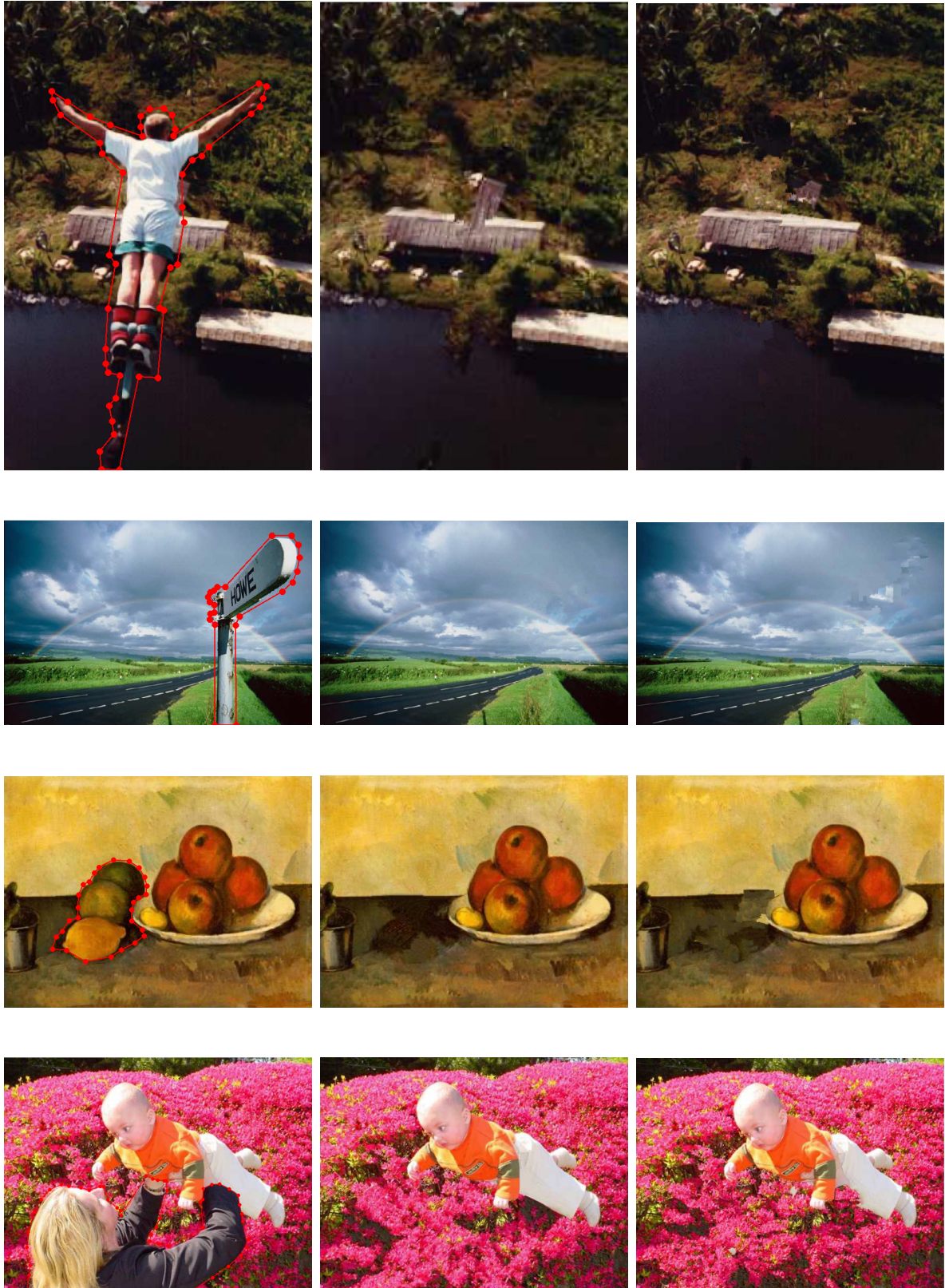


FIGURE 4 – De gauche à droite : image initial, résultat de l'article de Criminisi et résultat de notre algorithme (images tirées de [1] avec à la troisième ligne : “Still Life with Apples”, P. C´ezanne, c. 1890, The Hermitage, St. Petersburg.)

On remarque sur ces exemples que la qualité des remplissages de notre implémentation est proche de celle de Criminisi et al.[1]. L'avantage de notre implémentation est que la sélection

de la zone depuis laquelle l'algorithme copie les patches par l'utilisateur permet d'avoir plus de contrôle sur l'inpainting. Une bonne sélection de cette zone par l'utilisateur mène très souvent à un bon résultat. Cependant, l'algorithme que nous avons implémenté semble avoir plus de difficultés à reproduire des frontières nettes, c'est-à-dire prolonger précisément les isophotes de l'image.

Cela peut être dû à un manque de précision dans le calcul des gradients ou des vecteurs normaux au niveau de la frontière, le terme de données devenant alors moins précis.

Les résultats ci-dessus ont tous été obtenus en moins d'une minute, et il suffit même de quelques secondes pour obtenir le troisième résultat, si la zone des patches sources est choisie suffisamment petite.

5.2 Impact de la taille du patch

Les patches sont de forme carrée et leur taille peut varier. Elle doit nécessairement être un nombre impair de pixels afin de posséder un pixel central. La taille choisie pour les patches impacte significativement la qualité des résultats ainsi que la vitesse d'exécution de l'algorithme.

5.2.1 Impact sur la complexité temporelle

Une plus grande taille de patch augmente linéairement la complexité de nombreuses opérations telles que le calcul des priorités, la recherche de patches valides ou encore la mise à jour d'un patch. Toutefois, son influence se manifeste surtout de manière indirecte à travers le nombre d'itérations nécessaires à la complétion de l'algorithme.

En supposant qu'à chaque itération, le nombre de pixels ajoutés à la zone manquante est de l'ordre de P , alors le nombre total d'itérations pour remplir une zone de N pixels est en $\mathcal{O}(N/P)$. Autrement dit, ce nombre décroît linéairement avec la taille des patches. Cette décroissance se compense avec la croissance linéaire des autres opérations.

Ce qui signifie qu'augmenter la taille des patches n'a pas d'impact sur la complexité théorique de l'algorithme. Cependant en pratique on remarque qu'augmenter la taille des patches permet de réduire drastiquement le temps d'exécution.



FIGURE 5 – Image initiale utilisée pour comparer les temps d'exécution

TABLE 1 – Temps d'exécution en fonction de la taille du patch

Taille du patch (p) en pixel	Temps d'exécution
3	9 min
9	1 min 57 s
15	1 min 24 s
21	1 min 14 s

5.2.2 Qualité du résultat

La qualité du résultat est généralement meilleure avec des patches plus petits, car ils permettent une modification plus fine et localisée de l'image. Cependant, une taille de patch inférieure à la taille perceptible par l'œil humain n'apporte pas d'amélioration visuelle significative, tout en alourdissant inutilement les calculs. Il convient donc de choisir un compromis entre précision visuelle et performance algorithmique.



(a) Image initiale



(b) Résultat pour $p = 3$



(c) Résultat pour $p = 5$



(d) Résultat pour $p = 9$



(e) Résultat pour $p = 15$

FIGURE 6 – Comparaison de la qualité du résultat en fonction de la taille des patches (*image provenant de la banque d'image Vecteezy*)

6 Améliorations de l'algorithme

Plusieurs pistes d'améliorations peuvent être envisagées pour optimiser à la fois la qualité des résultats et le temps de calcul.

6.1 Espace colorimétrique CIE LAB plutôt que RGB

Dans l'ensemble du projet, les images sont représentées par la superposition de trois canaux de couleur (R, G, B). Toutefois, cette représentation n'est pas optimale pour le traitement d'image. En effet, dans l'espace RGB, la distance SSD (utilisée à l'étape (2)) n'est pas perceptuellement uniforme : deux couleurs qui semblent proches à l'œil humain peuvent être séparées par une distance SSD élevée. Pour pallier ce problème, il est possible de convertir les canaux RGB dans l'espace CIE Lab, dans lequel la distance SSD est plus cohérente avec la perception visuelle.

6.2 Approche par optimisation globale

L'approche gloutonne de l'algorithme de Criminisi peut conduire à des résultats visuellement incohérents, car elle ne prend pas en compte la cohérence globale entre les patches remplis au fil des itérations. Pour pallier cette limitation, Komodakis et Tziritas [5] ont proposé une méthode d'inpainting par patch fondée sur une optimisation globale. Contrairement à l'approche gloutonne, leur méthode évalue la cohérence visuelle du remplissage à l'aide d'une fonction objectif bien définie. L'algorithme cherche alors la configuration de patches qui minimise cette fonction, ce qui permet d'obtenir un résultat globalement plus cohérent.

Remercement

Nous souhaitons remercier Pascal Monasse pour son accompagnement tout au long de ce projet de recherche. Son expertise dans le domaine, ainsi que la qualité de ses cours d'informatique en C++, ont grandement contribué à la réalisation de notre travail.

Références

- [1] A. Criminisi, P. Pérez, and K. Toyama, *Region Filling and Object Removal by Exemplar-Based Image Inpainting*, IEEE Transactions on Image Processing, vol. 13, no. 9, pp. 1200–1212, 2004.
- [2] M. Bertalmío, A. L. Bertozzi, and G. Sapiro, "Navier-Stokes, fluid dynamics, and image and video inpainting," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Kauai, HI, USA, 2001, pp. I–I, doi : 10.1109/CVPR.2001.990497.
- [3] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, *Context Encoders : Feature Learning by Inpainting*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2536–2544.
- [4] M. Chen, Z. Liu, L. Ye, and Y. Wang, *Attentional Coarse-and-Fine Generative Adversarial Networks for Image Inpainting*, Neurocomputing, vol. 405, pp. 259–269, 2020, ISSN 0925-2312.
- [5] Nikos Komodakis and Georgios Tziritas. *Image Completion Using Global Optimization*. In *Proceedings of the 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2006)*, volume 1, pages 442–452, IEEE, 2006.